Predicting NFL Game Outcomes Using Machine Learning

Jaden Hyatt Emily Zapata Aditi Ravi Anastasiya Peravalava
Senait Pirani
STAT/CSCI 4990 — Spring 2026

1 Introduction

The legalization of sports betting in the US following the 2018 Supreme Court ruling in Murphy v. NCAA raised concerns about the integrity of the NFL [1]. Since then, the industry has grown rapidly, with the NFL embracing betting partnerships and integrating odds into broadcasts [2]. This shift raised a natural question: beyond changing how fans engage with the game, has the betting era affected the games outcomes? We had two primary questions. First is, can pregame data (betting lines, recent performance, weather) predict NFL game outcomes? Second is, Did scoring patterns shift after 2018, and do models perform differently in playoffs vs. regular season? By analyzing historical game data alongside betting market information, we can uncover whether sports betting has made outcomes more or less predictable [3]. Solving this problem would allow us to be able to analyze how sports betting has impacted game results, scoring patterns, and game strategies which can impact the way people bet on future games.

2 Data

2.1 Source and Variables

For our data, we utilized a Kaggle dataset that came from multiple sources such as ESPN/NFL.com scores, NOAA weather, and betting lines from multiple sources [4]. The dataset had 17 variables, with these being most important to our work: `schedule_date` (date the game was played), `schedule_playoff` (boolean indicating whether the game was a playoff game), `score_home` (final score of the home team), `score_away` (final score of the away team), `team_favorite_id` (abbreviated ID of betting favorite), `spread_favorite` (point spread assigned to the favored team), `stadium_neutral` (boolean indicating whether the game was played at a neutral site), `weather_temperature` (temperature in °F at game time), `weather_wind_mph` (windspeed in mph at game time), and `weather_humidity` (relative humidity % at game time). The dataset had games from 1966 to 2024; however, we only focused on games after 2018, after sports betting became legal.

2.2 Preprocessing

Prior to using the data in our analysis, we performed some important cleaning on the dataset. First, we changed the variable `schedule_date` to a datetime variable using pandas [5], making it easier to work with. Additionally, we sorted the dataset by `schedule_date` and `schedule_season`. There were quite a few missing values in this dataset, specifically in `weather_humidity`, `weather_wind_mph`, `weather_temperature`, `spread_favorite`, `team_favorite_id`, `weather_detail`, and `over_under_line`. There were only 5 NA values for `team_favorite_id` and `spread_favorite`, so we dropped these rows from the dataset. The variables `weather_detail` and `over_under_line` were very sparse with over half of the entries being NA, so we decided not to consider these variables when building the model.

For the remaining variables, we used imputation to replace the NA value with the median of that column. Lastly, we created a target variable to look at the percentages of home/away game wins in the dataset. This resulted in 57% home game wins and 43% away game wins. It is clear that there is a home field advantage, which will be considered in our conclusion.

2.3 Feature Engineering

From the raw data, we engineered three features to predict the outcome of the next game. The features added are rolling 5-game point differential, rolling home win rate, and home game favorite.

2.4 Exploratory Data Analysis

Exploratory analysis of scoring trends showed an increase of average total points per game from 41.81 before 2018 to 45.91 after 2018, and close games became more frequent. As shown in Figure 1, this rise in scoring is part of a long-term upward trend that predates betting legalization [6], so we cannot conclude that betting legalization caused the change. The most telling variable in the dataset is the `spread_favorite` distribution, which is closely tied to the outcomes [3].

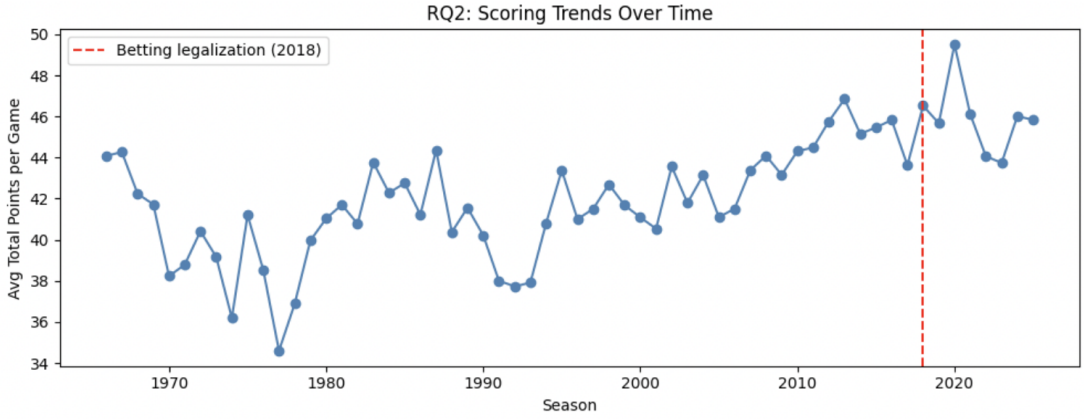


Figure 1: Average total points per game by NFL season (1966–2024). The dashed red line marks 2018 sports betting legalization. The upward trend predates 2018, suggesting the scoring increase is not caused by betting legalization.

3 Methods

3.1 Train/Test Split

Since NFL games are chronologically ordered, a random train/test split would not be suitable. If we randomly assigned games to training and testing sets, the model may be evaluated on a game that came before games it had been trained on, which would have given us an overly optimistic picture of performance. Instead, we split the data chronologically: we trained on games from 2018–2022 and held out games from 2023–2024 as the test set, ensuring that the model is always evaluated on games that took place after the training window.

We used `StandardScaler` to transform each feature to have zero mean and unit variance to allow us to use logistic regression. The scaler was fit only on the training set and applied to both the training and testing sets. If we had fit the scaler on the full dataset, information from the test set would have leaked into preprocessing. After dropping rows with remaining missing values in the engineered features, the final dataset contained 2,213 observations: 1,359 in training and 854 in the test set. Of the 854 test games, 815 were regular season and 39 were playoff games.

3.2 Logistic Regression

Logistic regression was used as our baseline model. Given a vector of pregame features $\mathbf{x}$, it models the probability that the home team wins as

$$P(\texttt{home_win} = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-(\beta_0 + \boldsymbol{\beta}^\top \mathbf{x})}}$$

The coefficients are estimated by minimizing the cross-entropy loss over the training set:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^{n} [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

We used the default L-BFGS solver [7] with a maximum of 1,000 iterations and default L2 regularization. L-BFGS is a quasi-Newton optimization method that converges reliably on datasets of this size and handles the L2 penalty without additional tuning. The key limitation here was the linearity assumption: the model assumes the log-odds of a home win is a linear combination of the input features, so it cannot learn interactions between features without manually constructing them.

3.3 Random Forest

Random Forest [8] is an ensemble method that builds a large number of decision trees and combines their predictions by majority vote. Each tree is trained on a bootstrap sample of the training data, and at each split a random subset of features is considered as candidates. The injection of randomness at the data and feature levels means that individual trees are deliberately different from each other. Averaging across many trees yields an ensemble with much lower variance than any single tree, which is the core idea behind bootstrap aggregation [9].

Each split is chosen to minimize the Gini impurity:

$$G(t) = 1 - \sum_{k} p_k^2$$

where p_k is the proportion of class k observations at node t. Random Forest can capture nonlinear relationships and feature interactions without manual specification, and it does not require feature scaling because tree splits depend only on the rank ordering of feature values.

We tuned three hyperparameters using `GridSearchCV` [10] with 3-fold cross-validation: `n_estimators` $\in \{100, 200\}$, `max_depth` $\in \{3, 5, \text{None}\}$, and `min_samples_split` $\in \{2, 5\}$, giving 12 candidate configurations. The best configuration was `max_depth` $= 3$, `min_samples_split` $= 2$, `n_estimators` $= 100$. The shallow depth suggests that deeper trees were overfitting rather than learning generalizable patterns.

3.4 XGBoost

XGBoost [11] is a gradient boosted tree method. Unlike Random Forest, which builds trees independently and in parallel, XGBoost builds them sequentially, with each new tree designed to correct the errors of the ensemble thus far. At each boosting round t, a new tree f_t is fit to the negative gradient of the loss from the current ensemble's predictions, and added with a learning rate η:

$$\hat{y}^{(t)} = \hat{y}^{(t-1)} + \eta \cdot f_t(\mathbf{x})$$

The full objective includes log-loss over training observations plus a regularization term $\Omega(f_t)$ penalizing tree complexity:

$$\mathcal{L} = \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] + \sum_t \Omega(f_t)$$

The regularization penalizes the number of leaves and the magnitude of leaf weights, which helps prevent overfitting on a training set spanning only five seasons. The learning rate η plays a complementary role in that smaller values require more trees but tend to generalize better.

We included XGBoost specifically because its tree structure supports SHAP analysis [12]. SHAP decomposes each individual prediction into additive contributions from each input feature by calculating the average marginal contribution of each feature across all possible orderings of features [12].

Hyperparameters were tuned via the same `GridSearchCV` procedure over `n_estimators` $\in \{100, 200\}$, `max_depth` $\in \{3, 5\}$, and `learning_rate` $\in \{0.05, 0.1\}$, giving 8 candidate configurations.

3.5 Hyperparameter Tuning

For both Random Forest and XGBoost, we used `GridSearchCV` [10] to evaluate all combinations of the hyperparameter values described above. For each combination, 3-fold cross-validation was applied within the training set: the data was divided into three folds, the model was trained on two and validated on the third, repeated so that each fold served as validation once. The combination with the highest average cross-validated accuracy was chosen. The test set was completely excluded from this process. Selecting hyperparameters by evaluating against the test set directly would produce biased accuracy estimates, an issue known as "peeking" [13]. By tuning via CV within the training window and evaluating only once on the held-out test set, the reported figures reflect actual out-of-sample performance.

4 Results

4.1 Model Comparison

Table 1 reports accuracy and log loss for all three models on the held-out 2023–2024 test set ($n = 854$ games). Random Forest achieved the highest accuracy at 62.5% with a log loss of 0.653. Logistic Regression performed nearly as well at 61.2% accuracy and 0.654 log loss. XGBoost underperformed both despite being the most expressive model, reaching 58.7% accuracy and a log loss of 0.685.

Table 1: Model performance on the held-out 2023–2024 test set ($n = 854$).

Model	**Accuracy**	**Log Loss**
Logistic Regression	61.2%	0.654
Random Forest	**62.5%**	**0.653**
XGBoost	58.7%	0.685

The close gap between logistic regression and Random Forest suggests that much of the predictive signal in the pregame features is approximately linear, and the additional complexity of the ensemble provides only a marginal gain. XGBoost's underperformance despite being the most complex is consistent with known behavior of gradient boosting on small datasets. With only five seasons of training data, the sequential boosting process overfits to training

patterns that do not generalize well to the test window [11]. All three models cluster in the low 60s, which is above the baseline of 57% from always predicting the home team as the winner, but well below what would be required for any practical application. This ceiling is consistent with prior work on pregame NFL predictions [14, 15] and reflects the variance introduced by factors in the game itself, including injuries, turnovers, and referee calls, that no pregame feature set can anticipate.

4.2 Feature Importance

Figure 2 presents the SHAP analysis computed using `TreeExplainer` [12] on the tuned XGBoost model. The left panel ranks features by mean absolute SHAP value across all 854 test games; the right beeswarm panel shows the direction and spread of each feature's contribution, with color indicating whether the feature value was high (pink) or low (blue) for a given observation.

The betting spread (`spread_favorite`) is the strongest predictor by a wide margin. Large absolute spread values consistently produce large SHAP magnitudes pointing toward the favored team, which is consistent with the efficiency of prediction markets [3]: oddsmakers collect rich information about team quality into a single number, and that number carries more signal than any feature we constructed from game logs alone. The rolling point differentials for the away team (`rolling_pd_away`) and home team (`rolling_pd_home`) rank second and third respectively, confirming that recent momentum adds signal beyond the spread. The beeswarm plot shows the expected directional pattern: teams winning by larger margins recently are associated with positive SHAP contributions in their favor. Rolling home win rate (`rolling_winrate_home`) contributes moderate importance. The three weather variables appear in the ranking but with low mean absolute SHAP values, indicating minimal average predictive value across the test set. `schedule_playoff` and `home_is_favorite` contribute essentially nothing because it showed zero variance in the dataset and was redundant with `spread_favorite`.

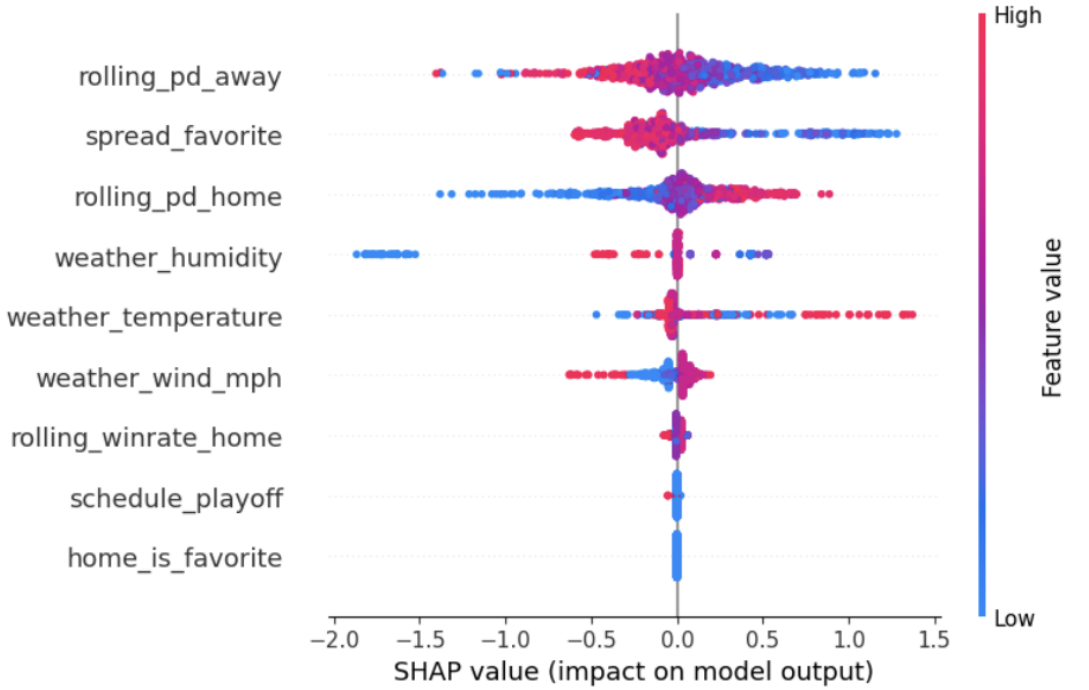


Figure 2: SHAP feature importance for the tuned XGBoost model (test set, $n = 854$).

4.3 Playoff vs. Regular Season Performance

To address whether our models perform differently in playoff games, we split the 854-game test set into 815 regular season games and 39 playoff games and computed accuracy and log loss separately for each subgroup. Table 2 reports the results.

Table 2: Model accuracy and log loss by game type (2023–2024 test set).

Model	Reg. Acc.	Playoff Acc.	Reg. Log Loss	Playoff Log Loss
Logistic Regression	61.0%	**66.7%**	0.653	0.680
Random Forest	**62.7%**	59.0%	**0.652**	**0.657**
XGBoost	59.4%	43.6%	0.676	0.880

The results differ substantially across models. Logistic Regression is the only model that performs better on playoff games (66.7%) than regular season games (61.0%), which may reflect its heavier weighting of the betting spread, a variable that is particularly informative in playoff matchups where oddsmakers have large amounts of information about team quality. Random Forest shows a modest decline in playoff accuracy (59.0% vs. 62.7%) but remains relatively stable. XGBoost's accuracy falls to 43.6% and log loss increases to 0.880. This pattern is consistent with overfitting to regular season dynamics and is exacerbated by the structural differences of playoff football, where teams are more evenly matched and game-to-game variance is higher [13].

The small playoff sample ($n = 39$) means that these estimates are noisy and should be interpreted as such. A difference of even a small number of games would change the accuracy by several percentage points, so the playoff results are better treated as suggestive than conclusive. A larger study incorporating multiple test seasons would be needed to draw firm conclusions about playoff predictability.

4.4 Super Bowl LVII Case Study

As an application, we applied the three trained models to predict the outcome of Super Bowl LVII (Philadelphia Eagles vs. Kansas City Chiefs, February 2023). Because the Super Bowl is played at a neutral site with no home team, we arbitrarily assigned the Eagles as the home team to satisfy the input structure of our models. This is an important limitation: our training data reflects a 57% home win rate, so this label assignment introduces a bias toward predicting the team designated as "home."

The pregame feature row (Table 3) was constructed from each team's rolling statistics over their five most recent games leading into the Super Bowl. The Eagles were stronger, with a rolling point differential of 9.4 versus 7.2 for the Chiefs and a perfect rolling win rate. The betting spread of -1.5 narrowly favored the Chiefs.

The three models disagreed in their predictions. Logistic Regression assigned a 43.7% win probability to the Eagles, predicting a Chiefs victory. Random Forest gave the Eagles 53.1%, predicting an Eagles win. XGBoost assigned 61.7% to the Eagles, also predicting an Eagles win. The actual result was a Chiefs victory, 38–35. Only Logistic Regression predicted correctly.

The disagreement reflects an issue between two signals in the feature row. The Eagles' stronger momentum features, combined with their "home" designation, pushed both ensemble models towards an Eagles prediction. Logistic Regression, which operates in a linear framework that weights the betting spread more directly, arrived at the correct call, though only with 56.3% confidence. The case study illustrates both the promise and limits of pregame-only NFL prediction: in a near-even matchup, small differences in how models weight market-derived

Table 3: Pregame feature row, Super Bowl LVII.

Feature	Value
`rolling_pd_home` (Eagles)	9.4
`rolling_pd_away` (Chiefs)	7.2
`rolling_winrate_home`	1.0
`home_is_favorite`	1.0
`spread_favorite`	−1.5
`weather_temperature`	72°F (indoor)
`weather_wind_mph`	0.0
`weather_humidity`	50%
`schedule_playoff`	1.0

features versus recent performance can flip the prediction, and the outcome remains largely uncertain regardless of which model is chosen.

5 Discussion

5.1 Evaluation of Results

The results show that it is inherently difficult to forecast NFL game outcomes using only pregame data [14, 15]. Although Random Forest achieved the highest accuracy at about 62.5%, the overall performance across all models was similar, which suggests that a large portion of game outcomes are influenced by factors not captured in the dataset. An important finding is the influence of the betting spread, which consistently appeared as the most important predictor. This demonstrates that the models are heavily dependent on preexisting information already ingrained within betting markets [3]. This suggests that rather than discovering new predictive patterns and relationships, the models may be replicating preexisting betting knowledge. However, recent team performance metrics like rolling point differential also had a big impact, emphasizing how crucial short-term momentum is in determining results [14].

5.2 Limitations

Despite these conclusions, the data still has important limitations. It is essential to compare various model types since doing so reveals the underlying relationships' structure. Logistic regression implies linear correlations between predictors and outcomes, whereas Random Forest [8] and XGBoost [11] can capture nonlinear interactions. Random Forest's slightly better performance indicates that the data does contain some nonlinear patterns and interactions. Even though the performance difference is small, it underscores the presence of nonlinear interactions and implies that the existing feature set's low complexity may prevent nonlinear models from fully utilizing it. Furthermore, several features—including the home team favorite indicator, which did not change between games—did not contribute to predictive accuracy.

Furthermore, because model findings were largely consistent across regular-season and postseason games, factors like weather and playoff status had minimal effect on overall forecast performance [12]. Significant missing data led to the removal of other potentially helpful variables, such as weather detail and the over/under line, which may have reduced the models' overall effectiveness. The assignment of a "home" team in a neutral-site game was one of the potential biases found in the Super Bowl case study. This would have affected forecasts because of the underlying home-field advantage [13]. Additionally, there is an

imbalance in the dataset, with the home team winning about 57% of games, indicating that the algorithms may have preferred predicting home team victories. Overall, even though the models capture some important structure, their reliance on betting-related factors and their modest advantage over nonlinear techniques indicate limited predictive ability on their own and highlight the need for richer, more informative features [14, 15]. Furthermore, it is challenging to determine how well these results will apply over a longer time period because the models were only tested on two seasons of test data (2023–2024). Additionally, XGBoost's lower performance indicates that the increased complexity may have caused overfitting rather than improving performance [11].

6 Conclusion

Understanding whether NFL game outcomes can be predicted using pregame data is important from a practical standpoint, particularly given the rapid growth of sports betting today [3]. As betting markets become more widespread, there is a demand for more data-driven approaches that utilize machine learning and statistics in order to make informed decisions that can provide a competitive edge. By evaluating the performance of predictive models, this study helps determine whether statistical and machine learning methods can meaningfully improve upon existing market expectations or if those markets already incorporate most of the available information [3]. This research contributes to the field of sports analytics by exploring the limitations of models and their prediction strengths [14, 15]. NFL games are influenced by a wide variety of factors, including player performance, coaching decisions, and other random events, many of which are difficult to quantify [6]. This allows for an interesting setting for testing models and researching this problem in order to better understand how well observable data can capture real-world outcomes and the need for more detailed features and advanced methods to improve predictive accuracy [15].

References

[1] Rodenberg, R. M. "Sports betting's legalization in the United States." *Journal of Legal Aspects of Sport* 29.1 (2019): 1–12.

[2] Drape, J., and Belson, K. "How the NFL embraced sports gambling." *New York Times*, 2022.

[3] Sauer, R. D. "The economics of wagering markets." *Journal of Economic Literature* 36.4 (1998): 2021–2064.

[4] Repole, S. "NFL Scores and Betting Data." *Kaggle*, 2024. `https://www.kaggle.com/datasets/tobycrabtree/nfl-scores-and-betting-data`

[5] McKinney, W. "Data structures for statistical computing in Python." *Proceedings of the 9th Python in Science Conference*. 2010.

[6] Lopez, M. J., Matthews, G. J., and Baumer, B. S. "How often does the best team win? A unified approach to understanding randomness in North American sport." *Annals of Applied Statistics* 12.4 (2018): 2483–2516.

[7] Liu, D. C., and Nocedal, J. "On the limited memory BFGS method for large scale optimization." *Mathematical Programming* 45.1 (1989): 503–528.

[8] Breiman, L. "Random forests." *Machine Learning* 45.1 (2001): 5–32.

[9] Breiman, L. "Bagging predictors." *Machine Learning* 24.2 (1996): 123–140.

[10] Pedregosa, F., et al. "Scikit-learn: Machine learning in Python." *Journal of Machine Learning Research* 12 (2011): 2825–2830.

[11] Chen, T., and Guestrin, C. "XGBoost: A scalable tree boosting system." *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016.

[12] Lundberg, S. M., and Lee, S.-I. "A unified approach to interpreting model predictions." *Advances in Neural Information Processing Systems* 30 (2017).

[13] Glickman, M. E., and Stern, H. S. "A state-space model for National Football League scores." *Journal of the American Statistical Association* 93.441 (1998): 25–35.

[14] Harville, D. A. "Predictions for National Football League games via linear-model methodology." *Journal of the American Statistical Association* 75.371 (1980): 516–524.

[15] Boulier, B. L., and Stekler, H. O. "Predicting the outcomes of National Football League games." *International Journal of Forecasting* 19.2 (2003): 257–270.